Análise de arquivo benigno ou malware

Share

codigo.py

PY

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Lendo o conteúdo do arquivo codigo.py

Vou começar lendo o arquivo de forma estática, sem executá-lo.

Ran 3 commands, read a file

Li o arquivo sem executá-lo. Adianto que ele é **malware**, um *downloader/loader* de primeiro estágio. A análise segue em ordem de execução.

1. Aparência geral. O script (SHA-256 `aff28bcd158d78c61ab79e2928e26b6dec6e6dde440cc06580e244a4cc455562`) não faz nada útil para o usuário. Tem funções e variáveis com nomes aleatórios (`fvzsrnzlh`, `scyzzvvy`, `uezupbxi`), emulação de `Buffer` e `__name` do JavaScript e um trecho morto (`if err` repetido no `callback`). Isso indica uma tradução mecânica de um código JS ofuscado para Python.

2. Atraso inicial. `main()` começa com `asyncio.sleep(10)`, uma evasão simples de sandboxes automáticas de análise rápida.

3. Geofencing. `_isRussianSystem()` olha usuário, `LANG` / `LANGUAGE` / `LC_ALL`, locale, fuso horário e deslocamento UTC. Se o idioma for russo e o fuso ou deslocamento também, o script encerra em silêncio. Excluir a própria região é prática comum em malware, mas não prova autoria, porque pode ser uma falsa pista.

4. Marcador `~/init.json`. Se o arquivo existir e tiver menos de 2 dias, o script não roda de novo. No macOS ele mesmo cria o arquivo. Isso limita a frequência da infecção e reduz ruído.

5. Servidor de comando via blockchain. `fvzsrnzlh()` consulta a carteira Solana `BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC` em cerca de 9 nós RPC públicos, alguns com chaves de API embutidas. Repete a cada 10 segundos até achar uma transação com campo `memo`, que traz um JSON com um `link` em base64. O atacante troca o servidor a qualquer momento publicando um novo memo, e o tráfego parece legítimo. É uma técnica conhecida como *dead drop resolver*.

6. Download do payload. `scyzzvvy()` decodifica o link e faz um GET enviando o cabeçalho `os` com o sistema operacional da vítima. Os nomes dos cabeçalhos de resposta

estão escondidos em base64 e decodificam para `ivbase64` e `secretkey`, ou seja, IV e chave de criptografia enviados pelo servidor.

7. Baixa o Node.js. `download_nodejs()` baixa o Node v22.9.0 de nodejs.org para a pasta pessoal e extrai. A fonte é legítima, mas serve para garantir um interpretador JS na máquina sem depender de nada instalado.

8. Execução do código remoto. O script grava `i.js` ao lado dele, com `secretKey`, `_iv` e `eval(atob('<payload>'))`. Depois roda com `subprocess.Popen`, sem janela no Windows e com saída descartada. É execução remota de código arbitrário, e o atacante decide em tempo real o que ela faz.

Limitações. O payload final não está no arquivo, e eu não tentei baixá-lo. Por isso não dá para dizer se é infostealer, backdoor ou outra coisa. O padrão de chave e IV sugere um próximo estágio criptografado, mas isso é inferência. Também não consultei bases de reputação, então vale checar o hash no VirusTotal. Nada disso muda o veredito.

Conclusão

É malware, com confiança alta. Ele combina evasão de análise, geofencing, comando via blockchain, fingerprint do sistema, download de runtime próprio e execução oculta de código remoto cifrado. Não existe uma leitura benigna plausível para essa combinação.

O que fazer:

- **Não execute o arquivo.** Se alguém ou algum repositório (por exemplo, um "teste técnico") te enviou, trate a origem como suspeita.
- **Se já executou**, desconecte a máquina da rede e procure por estes indícios:
  - `~/init.json`
  - `i.js` na pasta do script
  - a pasta `~/node-v22.9.0-*`
  - processos `node` que você não iniciou
- **Troque senhas, tokens e chaves** (navegador, SSH, GitHub, nuvem) a partir de **outro dispositivo limpo.** Se houver carteiras de criptomoedas na máquina, mova os fundos para uma carteira nova, com nova seed.
- **Considere reinstalar o sistema**, já que o que o payload final fez é desconhecido.

Qual é a categoria principal deste malware?

Distinguindo downloader de dropper ▸

Downloader/loader (primeiro estágio).

O script não faz o dano final. Ele busca um payload num servidor controlado pelo atacante e o executa (`eval` via Node.js). Por isso o termo mais preciso é *downloader*, e *loader* também se aplica porque ele executa o que baixa. Não é um *dropper* no sentido estrito, porque o payload não vem embutido no arquivo.

Características secundárias:

- **Resolução de C2 via blockchain** (*dead drop resolver* na Solana).
- **Evasão e guardrails:** atraso inicial, geofencing contra sistemas russos e marcador de execução.
- **Execução oculta:** processo em segundo plano, sem janela e sem saída.

A categoria do malware completo, como stealer, RAT ou ransomware, só dá para definir com o payload do segundo estágio, que não está neste arquivo. O que dá para dizer é que o impacto potencial é amplo, porque o código executado é decidido remotamente pelo atacante.